

TripMate

AI-Powered Collaborative Trip Planner

PHASE 3 – 5

Implementation Report

Authentication • User Profile & Identity Verification • Trip Planning & Collaboration

Document Type	Implementation Report
Based On	Phase 2 — Software Design Document (SDD)
Phases Covered	Phase 3, Phase 4, Phase 5
Architecture	MERN Stack (MongoDB, Express, React, Node.js)
Status	Backend Implementation Complete — Frontend Integration In Progress

Table of Contents

1. Introduction
2. Implementation Summary
3. Phase 3 — Authentication Module
4. Phase 4 — User Profile & Identity Verification Module
5. Phase 5 — Trip Planning & Collaboration Module
6. Backend Folder & File Structure
7. Testing Summary
8. Deviations From Phase 2 Design
9. Next Steps (Phase 6 Onward)
10. Conclusion

1. Introduction

1.1 Purpose of this Document

This document reports on the implementation status of Phase 3, Phase 4, and Phase 5 of TripMate, as defined in the Phase 2 Software Design Document (SDD). It records the features built, the API surface implemented, the database collections in active use, and the folder structure of the backend codebase. This report accompanies the corresponding GitHub commit as an implementation-tracking artifact for these three phases.

1.2 Scope

This report covers three consecutive implementation phases that build directly on the Phase 2 design: Authentication, User Profile & Identity Verification, and Trip Planning & Collaboration. Backend implementation for all three modules is complete and functional against the schema and API contracts defined in Phase 2. Frontend screens for these modules are being built in parallel; the complete frontend codebase will be added to the repository once development across all planned modules is finished.

1.3 Reference Documents

- Phase 1 — Software Requirement Specification (SRS)
- Phase 2 — Software Design Document (SDD): system architecture, module design, database schema, API design

2. Implementation Summary

The table below summarizes the phase-wise implementation status against the module design defined in the Phase 2 SDD.

Phase	Module	Status	Key Deliverables
Phase 3	Authentication	● Complete	Register, login, logout, session handling, email OTP verification, bcrypt password hashing
Phase 4	User Profile & Identity Verification	● Complete	Profile CRUD, document upload, verification workflow (Unverified → Pending → Verified)
Phase 5	Trip Planning & Collaboration	● Complete	Trip CRUD, join requests, member accept/reject, TripMembers role management

All three modules were implemented in the dependency order recommended in Phase 2 §12 (Conclusion & Next Steps): Auth and the core Trip module first, since every later module — collaboration, AI itinerary generation, chat, expenses — depends on an authenticated user and an existing trip.

3. Phase 3 — Authentication Module

3.1 Overview

The Authentication module was implemented exactly as scoped in Phase 2 §9 (Security Design): server-side sessions using **express-session** backed by **connect-mongo** so sessions survive server restarts and scale statelessly across instances, password hashing with **bcrypt**, and email-based OTP verification sent through **Nodemailer**. Routes follow the MVC split defined in Phase 2 §3.3 — thin route handlers, business logic in controllers, schema access through Mongoose models.

3.2 Features Implemented

- User registration with email, password, and mandatory profile fields
- Login with email/password, creating a persistent server-side session
- Logout with full session destruction
- Email OTP generation and verification for new accounts
- Password hashing via bcrypt before every write to **Users.passwordHash**
- Session persistence via connect-mongo, decoupling session state from any single server instance
- Rate limiting on all authentication endpoints to reduce brute-force and OTP-spam risk

3.3 API Endpoints

Method	Endpoint	Description	Status
POST	/api/auth/register	Register a new user account	● Implemented
POST	/api/auth/login	Validate credentials, create session	● Implemented
POST	/api/auth/logout	Destroy the active session	● Implemented
POST	/api/auth/verify-email	Confirm the emailed OTP code	● Implemented
POST	/api/auth/resend-otp	Reissue OTP on expiry/request	● Implemented

3.4 Security Implementation

Concern	Implementation	Status
Authentication	express-session + connect-mongo — persistent, restart-safe sessions	● Implemented
Password Storage	bcrypt hashing; plaintext password never persisted	● Implemented
Transport Security	HTTPS enforced; cookies set httpOnly and secure	● Implemented
Input Validation	Schema-level validation in controllers before every DB write	● Implemented
Abuse Prevention	Rate limiting applied to auth endpoints	● Implemented

Concern	Implementation	Status
HTTP Hardening	Helmet middleware for secure response headers	 Implemented

3.5 Database Fields in Use (Users collection)

Authentication reads/writes the following fields on the **Users** collection defined in Phase 2 §5.2: **email**, **passwordHash** (bcrypt), **isEmailVerified**, **otpCode** / **otpExpiresAt** (transient, cleared after verification), and session metadata persisted separately in the connect-mongo session store rather than embedded in the user document.

4. Phase 4 — User Profile & Identity Verification Module

4.1 Overview

This phase implements the two modules Phase 2 §4 groups closely together: User Management (profile) and Identity Verification. Profile photo and identity documents are uploaded to **Cloudinary** as scoped in the Phase 2 tech stack, with only the returned secure URLs stored in MongoDB.

4.2 Features Implemented

- View and edit profile: name, profile photo, age, gender, blood group, phone number, emergency contact
- Optional medical-conditions field, surfaced only in emergency-context screens per the Phase 2 design intent
- Vehicle information capture, used later for trip vehicle matching
- Profile photo upload/replacement via Cloudinary, with old asset cleanup
- Identity document upload flow, storing document URL and metadata in a dedicated VerificationDocuments record
- Verification status field on Users, updated as documents move through review
- Password recovery flow (forgot password → emailed reset link → new password)

4.3 API Endpoints

Method	Endpoint	Description	Status
GET	/api/users/:id/profile	Fetch a user's profile	● Implemented
PUT	/api/users/:id/profile	Update editable profile fields	● Implemented
POST	/api/users/:id/profile-photo	Upload/replace profile photo	● Implemented
POST	/api/users/:id/verification-documents	Submit identity document for review	● Implemented
GET	/api/users/:id/verification-status	Get current verification status	● Implemented
POST	/api/auth/forgot-password	Request a password-reset email	● Implemented
POST	/api/auth/reset-password	Set a new password from reset token	● Implemented

4.4 Verification Workflow

The verification state machine follows Phase 2 §4 exactly: a user's **verificationStatus** starts at **Unverified**, moves to **Pending** the moment a document is submitted, and is manually reviewed against the uploaded document before being set to **Verified** (or returned to **Unverified** with a rejection reason if the document does not pass review). Downstream modules — Reviews & Ratings and trip-organizer trust signals — read this single status field rather than querying VerificationDocuments directly, keeping the coupling one-directional as designed.

4.5 Database Fields in Use

Collection	Fields in Active Use
Users	verificationStatus, bloodGroup, emergencyContact, medicalConditions, vehicleInfo, profilePhoto
VerificationDocuments	userId (ref), documentUrl, documentType, submittedAt, reviewStatus, reviewedAt, rejectionReason

5. Phase 5 — Trip Planning & Collaboration Module

5.1 Overview

This phase implements the Trip Planning and Collaboration modules from Phase 2 §4, covering trip lifecycle management and the join-request-based membership model. The AI Trip Planner, Real-Time Chat, and Map Integration modules that Phase 2 lists as consumers of this module's data are scoped for Phase 6 and are not part of this report.

5.2 Features Implemented

- Create, edit, and delete a trip (destination, budget, date, duration, vehicle, max members, difficulty level)
- Trip listing and detail views, including organizer-only edit/delete permissions
- Send a join request to an existing trip
- Organizer accept/reject actions on incoming join requests
- Member list per trip with role (organizer vs member) and membership status
- Leave-trip action for existing members

5.3 API Endpoints

Method	Endpoint	Description	Status
POST	/api/trips	Create a new trip	● Implemented
GET	/api/trips/:id	Get trip details	● Implemented
PUT	/api/trips/:id	Edit trip (organizer only)	● Implemented
DELETE	/api/trips/:id	Delete trip (organizer only)	● Implemented
POST	/api/trips/:id/join	Send a join request	● Implemented
PUT	/api/trips/:id/members/:memberId	Accept / reject a join request	● Implemented
DELETE	/api/trips/:id/members/:memberId	Remove member / leave trip	● Implemented
GET	/api/trips/:id/members	List trip members	● Implemented

5.4 Collaboration Workflow

Membership follows the TripMembers-as-join-table pattern from Phase 2 §5: a join request creates a TripMembers record with status **Requested**; the organizer's accept action sets it to **Approved**, and reject sets it to **Rejected**. Only Approved members appear in member-facing views such as the group discussion panel and, in Phase 6, the shared chat room.

5.5 Database Fields in Use

Collection	Fields in Active Use
Trips	destination, budget, date, duration, vehicle, maxMembers, difficultyLevel, organizerId
TripMembers	tripId (ref), userId (ref), role, status (Requested / Approved / Rejected), joinedAt

6. Backend Folder & File Structure

The backend implementation follows the folder layout proposed in Phase 2 §7.1, with routes, controllers, and models added per module as each phase was completed.

```
tripmate/
├── backend/
│   ├── config/
│   │   ├── db.js
│   │   ├── middleware/
│   │   │   ├── errorHandler.js
│   │   │   ├── authMiddleware.js
│   │   │   └── rateLimiter.js
│   │   └── models/
│   │       ├── User.js
│   │       ├── VerificationDocument.js
│   │       ├── Trip.js
│   │       └── TripMember.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── userController.js
│   │   ├── verificationController.js
│   │   └── tripController.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── userRoutes.js
│   │   └── tripRoutes.js
│   ├── services/
│   │   ├── emailService.js
│   │   └── cloudinaryService.js
│   └── server.js
```

Figure 6.1 — Backend folder structure as implemented through Phase 5.

7. Testing Summary

Each module was tested manually against its API contract before being marked complete, covering both expected and edge-case inputs.

Module	Method	Cases Covered	Result
Authentication	Manual API testing (Postman)	Duplicate email, wrong password, expired/invalid OTP, session persistence across restarts	● Passed
Profile & Verification	Manual API testing (Postman)	Invalid document type, oversized upload, status transitions, unauthorized profile edits	● Passed
Trip & Collaboration	Manual API testing (Postman)	Non-organizer edit/delete attempts, duplicate join requests, over-capacity trips	● Passed

Automated test coverage (Jest/Supertest) is planned alongside the Phase 6 rollout, once the API surface for AI itinerary, chat, and expenses is finalized.

8. Deviations From Phase 2 Design

No structural deviations from the Phase 2 design were required. Two endpoints not explicitly enumerated in Phase 2 §6 were added during implementation to complete the module's own workflow:

/api/auth/resend-otp and **/api/auth/forgot-password / reset-password**. Both extend, rather than alter, the Authentication design and are documented in §3.3 and §4.3 above.

9. Next Steps (Phase 6 Onward)

With Auth, Profile/Verification, and Trip/Collaboration complete, implementation proceeds to the remaining modules identified in Phase 2 §1.2, in the dependency order set out there:

- **AI Trip Planner** — Gemini API integration for itinerary, budget, and packing-list generation
- **Real-Time Chat** — Socket.IO chat rooms scoped to Approved trip members
- **Map Integration** — navigation and nearby-services lookup (Google Maps / OpenStreetMap)
- **Expense Management** — shared expense entry and auto-splitting
- **Emergency Module** — SOS trigger, first-aid guide, nearby emergency resources
- **Offline Support** — Service Worker + IndexedDB caching of critical trip and emergency data
- **Notifications and Reviews & Ratings** modules

The complete frontend codebase, currently being built against the endpoints documented in this report, will be committed once all modules above reach the same implementation status as Phase 3–5.

10. Conclusion

Phase 3, Phase 4, and Phase 5 are complete: the backend now supports full account authentication, profile management with a document-based identity verification workflow, and end-to-end trip creation with request-based collaboration — all implemented against the schema, API contracts, and security design finalized in Phase 2. This forms a stable foundation for the AI, real-time, and safety-oriented modules planned for Phase 6.

End of Phase 3–5 — Implementation Report